EXAMEN PRATIQUE DE REQUÊTES

420-2B4-VI Bases de données 1 **| Formatif**

Date : 20 février 2026

Durée : 50 minutes (correction à la 2ème heure)

Pondération : 0 %

Total : 27 points

Habiletés évaluées :

- Formuler les instructions de création de la base de données
- Déterminer les types de requêtes à formuler
- Utiliser les clauses, les opérateurs, les commandes ou les paramètres
- Utiliser les expressions régulières
- Utiliser les contraintes d'intégrité référentielle

Contexte de réalisation : individuel, répondre sur le questionnaire

Matériel autorisé : grille de correction

Matériel fourni : feuille aide-mémoire REGEX et SQL, modèle de BD,

Consignes lors de l'examen

- Répondez à toutes les questions en utilisant une écriture lisible.
- Les requêtes SQL doivent respecter les standards de programmation établis (indentation, retours à la ligne et majuscules).
- Supposez en tout temps que les structures de données requises existent pour l'exécution des requêtes (sauf lorsqu'on vous demande de créer lesdites structures).
- Toute forme de tricherie met fin immédiatement à l'examen, entraîne la note de 0 et un report au département.
- La répartition des points pour chaque question est écrite à côté de chaque énoncé.

PARTIE 1 — CRÉATION ET MANIPULATION DE STRUCTURES

6 points

1) Écrivez la requête pour créer la table suivante :

   Considérez que les clés primaires non composées de type entier doivent être auto-incrémentées.

   a) Table Auteurs (**4 points**)

2) Écrivez la requête pour modifier la structure afin de :

   a) Ajouter une colonne ville de naissance de type *varchar* permettant de saisir un maximum de 100 caractères à la table Auteurs (**2 points**)

PARTIE 2 — CRÉATION, MISE À JOUR ET SUPPRESSION DE DONNÉES

6 points

3) Écrivez les requêtes pour effectuer les manipulations suivantes sur les données :

   a) Ajouter un livre nommé « Catch 22 » de l'auteur avec l'identifiant 88 et de l'éditeur avec l'identifiant 43, publié en 1943. Ce livre est un ouvrage de fiction. (**3 points**)

   b) Le membre 334 a retourné le livre 7755 le 8 septembre 2023 pour l'emprunt 543322. Veuillez mettre à jour l'emprunt avec la date de retour. (**3 points**)

PARTIE 3 — SÉLECTION DE DONNÉES

11 points

4) Votre bibliothécaire Ginette a plusieurs demandes pour vous. Écrivez les requêtes pour répondre aux demandes de Ginette.

   a) Ginette souhaite récupérer les noms et les prénoms de l'ensemble des membres dont le nom débute par un T (**3 points**)

   b) Ginette souhaite connaître les 3 auteurs avec le plus de livres à la bibliothèque. (**4 points**)

   c) Ginette souhaite connaître le nom et le prénom du membre qui a un livre en emprunt et qui n'est pas retourné. (**4 points**)

PARTIE 4 — CORRECTION DE REQUÊTES

4 points

5) Ginette a écrit des requêtes de BD. Toutefois, elle n'est pas très habile avec le langage SQL et elle a commis quelques erreurs. Dans chacune des requêtes écrites par Ginette, identifiez les erreurs et réécrivez la requête en tenant compte de ces modifications. Chaque requête comporte deux erreurs (la casse et la mise en forme des requêtes sont toujours adéquates).

a) Sélectionner le prénom, le nom des membres, le nom du livre et le nombre de jour de retard pour les emprunts qui sont en retard (un emprunt est en retard lorsqu'il est emprunté plus de 10 jours) (**4 points**)

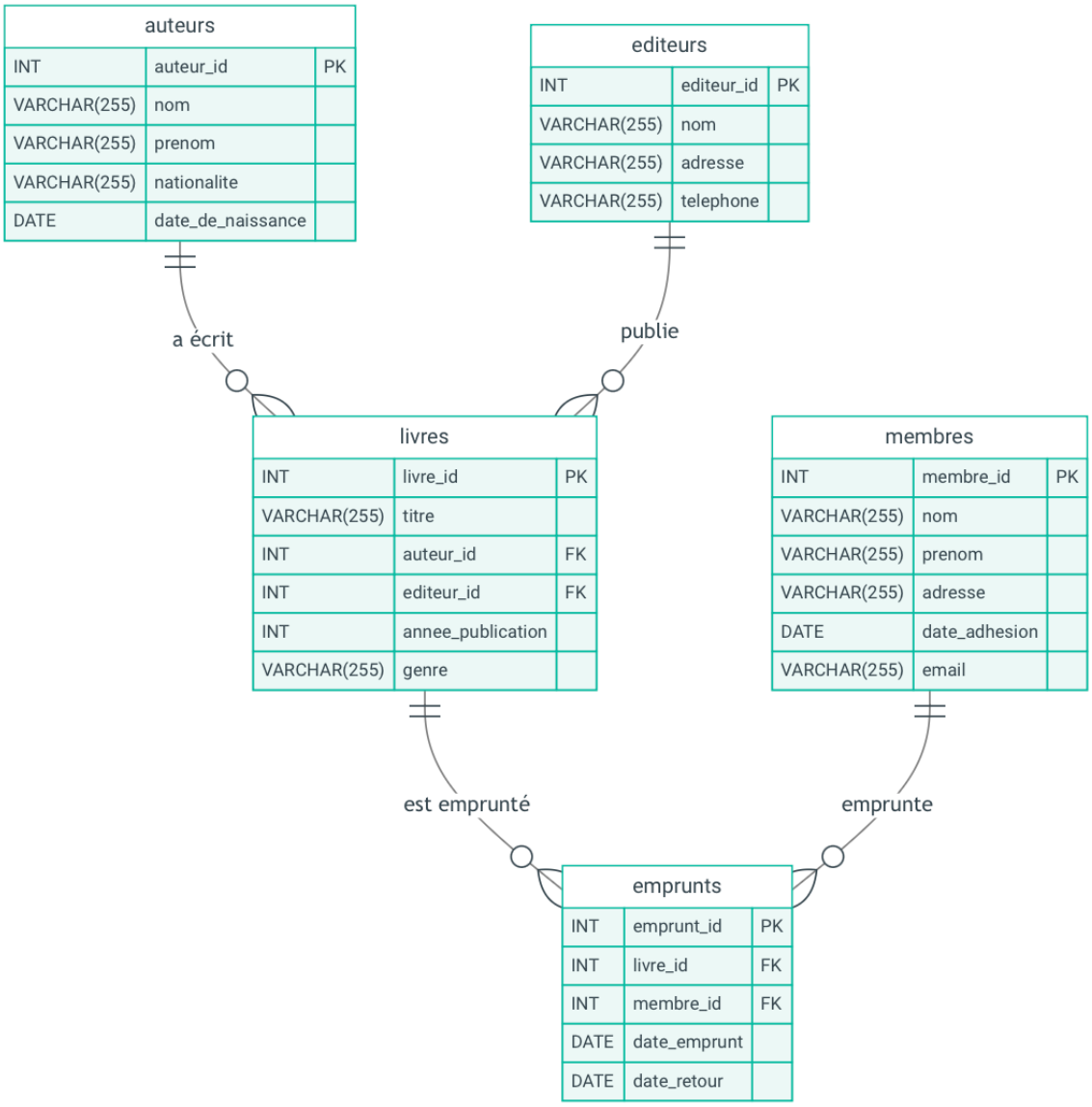
```
SELECT prenom, nom, titre,
    datediff(now(), adddate(date_emprunt, INTERVAL 10 DAY)) as jours_de_retard
    FROM emprunts
    INNER JOIN membres ON membres.membre_id = emprunts.membre_id
    INNER JOIN livres ON livre_id = livre_id
    WHERE date_retour = NULL AND
    datediff(now(), adddate(date_emprunt, INTERVAL 10 DAY)) > 0;
```

Erreur 1 :

Erreur 2 :

Requête corrigée :

DIAGRAMME DE BASE DE DONNÉES



Feuille de *triche* REGEX

Début et fin

^ : Début d'une chaîne
$: Fin d'une chaîne

Quantificateur

***** : 0, 1 ou plusieurs fois
? : 0 ou 1 fois
+ : 1 ou plusieurs fois
{p, q} : entre p et q fois
{p,} : au moins p fois
{,q} : au plus q fois
{p} : exactement p fois

Classes de caractères

[a-z] : toutes les lettres minuscules
[abcd] : n'importe quelle lettre dans les crochets
[a-zà-ÿA-ZÀ-Ÿ0-9] : N'importe quelle lettre, nombre ou lettre accentuée
[^a] : n'importe quoi sauf a
\w : tous les caractères alphanumériques (**[a-zA-Z0-9_]**)
\W : équivalent à **[^\w]**
\s : tous les caractères d'espace (**[\n\r\t]**)
\S : équivalent à **[^\s]**
\d : tous les caractères numériques (**[0-9]**)
\D : équivalent à **[^\d]**
. : n'importe quel caractère
- : littéral du trait d'union, il doit être le dernier caractère dans une classe
**** : permet d'échapper un caractère (exemple : **\.** pour rechercher le caractère point)
| : opérateur logique OU

Classes de caractères de MySQL

[:alnum:] : tous les caractères alphanumériques (**[a-zA-Z0-9]**)
[:alpha:] : toutes les lettres (**[a-zA-Z]**)
[:blank:] : tous les caractères blancs, mais pas les retours à la ligne (**[\t]**)
[:cntrl:] : tous les caractères de contrôle comme les tabulations ou les retours à la ligne
[:digit:] : tous les nombres (**[0-9]**)
[:lower:] : toutes les lettres minuscules
[:space:] : tous les caractères d'espace (semblable à **\s**)
[:upper:] : toutes les lettres majuscules

Langage de définition des données

```
CREATE DATABASE nom_bd;

CREATE TABLE nom_table (
   nom_id TYPE PRIMARY KEY AUTO_INCREMENT,
   nom_col2 TYPE,
   . . .
    FOREIGN KEY (nom_col2) REFERENCES autre_table (autre_table_id));

ALTER TABLE nom_table
   ADD COLUMN nouvelle_col TYPE,
   MODIFY COLUMN nom_col TYPE,
   DROP COLUMN nom_col;

DROP TABLE nom_table;
```

Langage de manipulation des données

```
INSERT INTO nom_table (nom_colonne1, nom_colonne2,...)
   VALUES (valeur_colonne1, valeur_colonne2,...);

UPDATE nom_table
   SET nom_colonne1 = valeur1, nom_colonne2 = valeur 2, ...
   WHERE condition;

DELETE FROM nom_table
   WHERE condition;

SELECT DISTINCT nom_colonne, nom_colonne2  AS renomme_col
   FROM nom_table
      INNER JOIN nom_table2 ON nom_table.col1 = nom_table2.col1
   WHERE condition AND NOT condition2 OR condition3
   GROUP BY nom_colonne
   HAVING condition
   ORDER BY nom_colonne DESC I ASC
   LIMIT 1, 5;
```

Conditions: `a = b, a > b, a < b, a LIKE 'T%', a RLIKE '[a-zA-Z]{1,2}.*'`

Formats : `Date: 'AAAA-MM-JJ', Heure: 'HH:MM:SS',`
`DateHeure: 'AAAA-MM-JJ HH:mm:SS'`

Types de données

NUMÉRIQUES
TINYINT - SMALLINT - MEDIUMINT - INT - BIGINT
FLOAT - DOUBLE - NUMERIC(x,y)
TEMPORELS
DATE - TIME - DATETIME - TIMESTAMP - YEAR
ALPHANUMÉRIQUES
CHAR(999) - VARCHAR(999) - TEXT - BLOB - ENUM

Fonctions

EXTRACTIONS TEMPORELLES
YEAR() - MONTH() - DAY() - HOUR() - MINUTE() - SECOND() - DAYOFWEEK() - DAYNAME() - MONTHNAME() - DAYOFYEAR() - WEEKOFYEAR() - WEEK()

DATE ET HEURE ACTUELLE
NOW() - CURRENT_DATE() - CURRENT_TIME() - CURRENT_TIMESTAMP()

MANIPULATIONS TEMPORELLES
ADDDATE(date, interval) - SUBDATE(date, interval) - DATEDIFF(date1, date2) - ADDTIME(heure1, heure2) - TIMESTAMPADD(unite, quantité, timestamp) - TIMESTAMPDIFF(unite, timestamp1, timestamp2)

INTERVALS TEMPORELS
INTERVAL 1 DAY - INTERVAL 1 WEEK - INTERVAL 1 MONTH - INTERVAL 1 YEAR

FONCTIONS STATISTIQUES
SUM(nom_col) - MIN(nom_col) - MAX(nom_col) - COUNT(nom_col) - AVG(nom_col) - STD(nom_col)

OPERATEURS D'ENSEMBLES
nom_col BETWEEN valeur AND valeur2 -
nom_col IN (valeur1, valeur2, ...)